



Ingegneria e Sicurezza del Software

Docente: **Simona E. Rombo**

Corso di Laurea in Informatica

Università degli Studi di Palermo

Design Pattern Creazionali

- Rendono astratto il processo di istanziare le classi
- Servono a rendere un sistema indipendente da come i suoi oggetti vengono creati, composti e rappresentati
- Uso spinto dell'ereditarietà per variare le classi che devono essere istanziate
- Incapsulano conoscenza su quali classi concrete sono utilizzate dal sistema
- Nascondono in che modo tali classi vengono create e messe insieme
 - Tutto ciò che il sistema vede sono le interfacce degli oggetti, definite nelle classi astratte

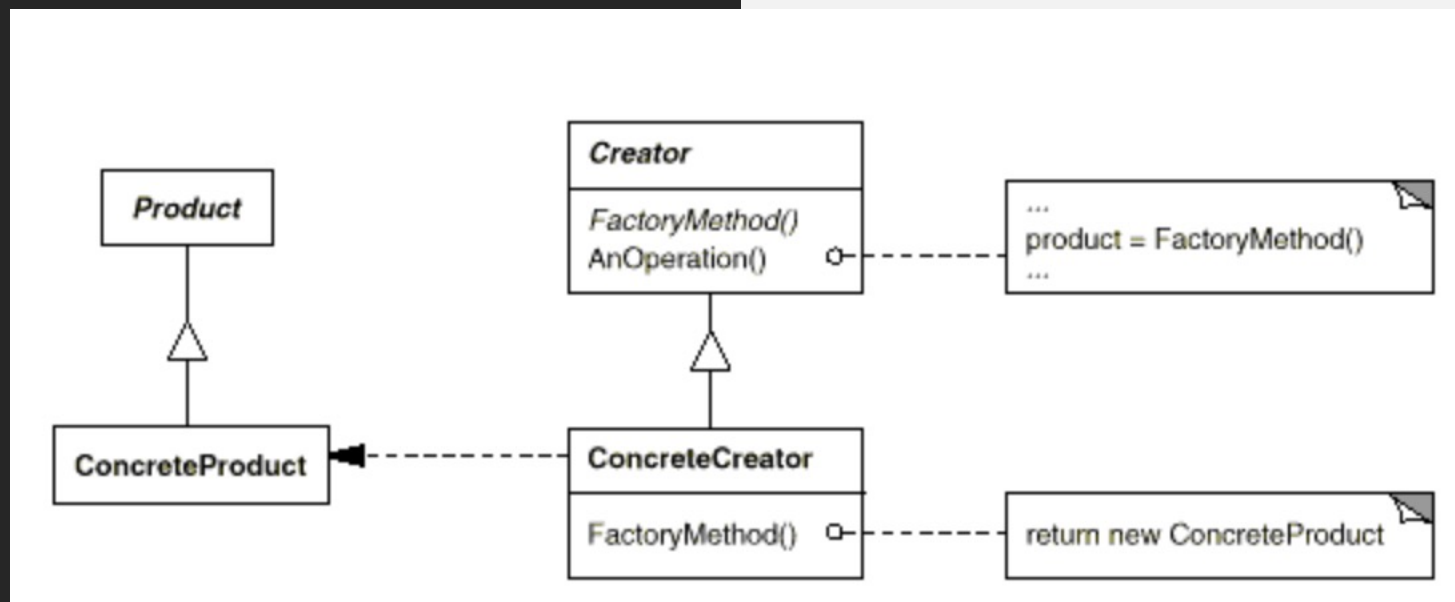
Design Pattern Creazionali

- Consentono ampia flessibilità in:
 - **What** – cosa viene creato
 - **Who** – chi lo crea
 - **How** – come viene creato
 - **When**
- Permettono di configurare un sistema con oggetti «prodotto» che variano ampiamente in struttura e funzionalità
- La configurazione può essere:
 - **Statica**, compile-time
 - **Dinamica**, run-time

Ancora su Factory Method

Da utilizzare quando:

- Una classe non può sapere in anticipo la classe di oggetti che deve creare
- Una classe vuole che siano le sue sottoclassi a specificare gli oggetti da creare
- Le classi delegano la responsabilità ad una o più sottoclassi «aiutanti», si vuole localizzare la conoscenza di quale sottoclasse aiutante è il delegato

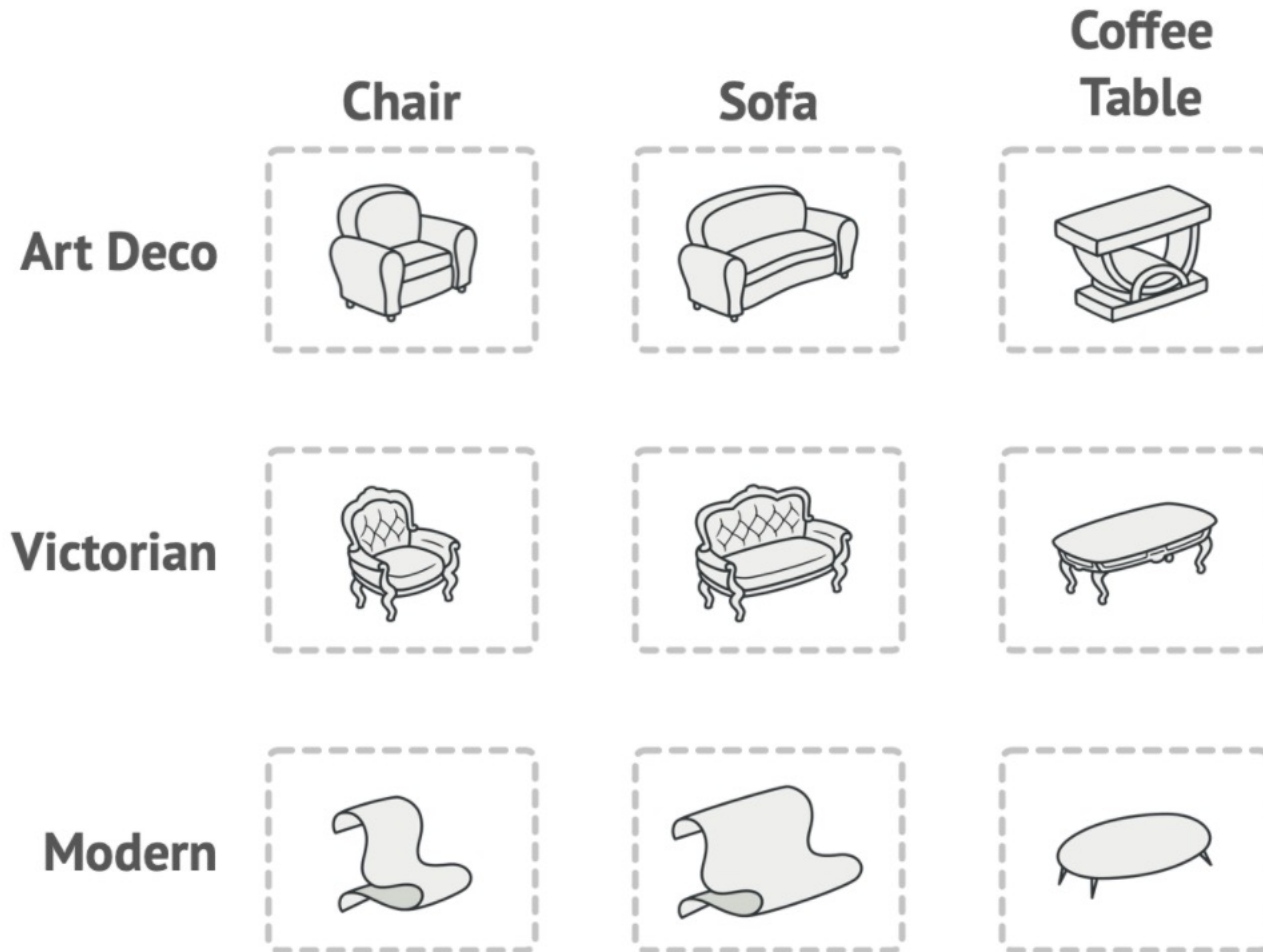


Abstract Factory

-
- Scopo
 - Fornire un'interfaccia per creare famiglie di oggetti correlati o dipendenti senza specificare le loro classi concrete
 - Applicabilità – Usa questo pattern quando:
 - Un sistema deve rimanere indipendente dal modo in cui i suoi prodotti vengono creati, composti e rappresentati
 - Un sistema deve essere configurato attraverso una o più famiglie di prodotti
 - Degli oggetti prodotto correlati e appartenenti alla stessa famiglia devono essere utilizzati insieme
 - Si vuole fornire una classe che rappresenti una libreria di prodotti, rivelando solo la loro interfaccia, non la loro implementazione

Abstract Factory

- Problema
 - Simulatore per un negozio di mobili
 - Il codice è composto da classi che rappresentano:
 - Una famiglia di prodotti correlati: Chair + Sofa + CoffeeTable
 - Diverse varianti di questa famiglia: Modern, Victorian, ArtDeco



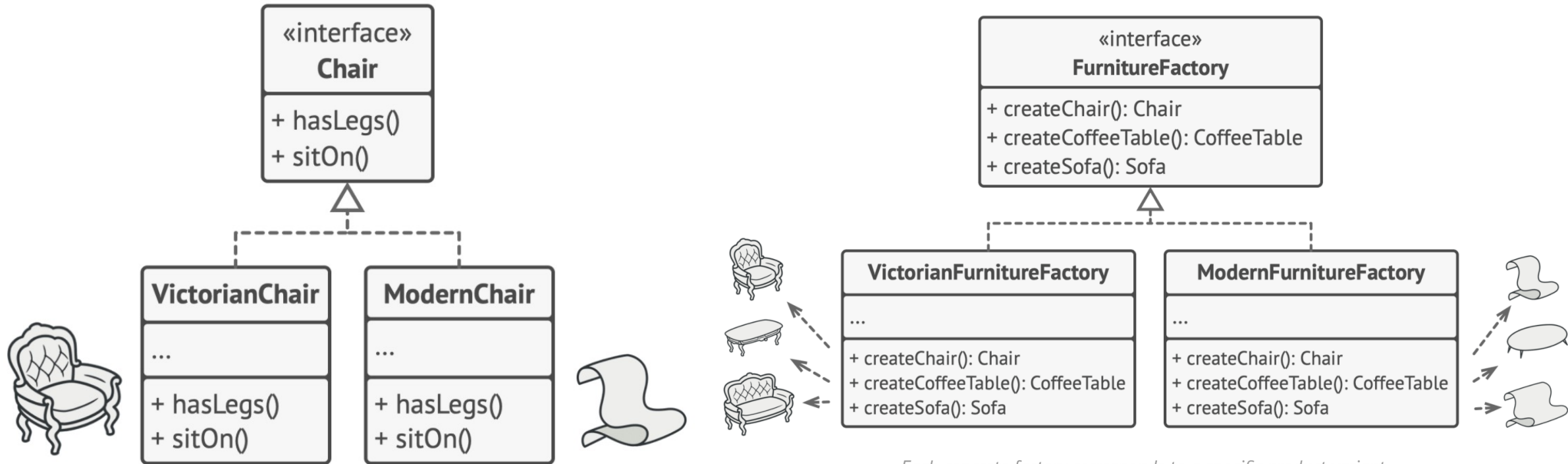
Product families and their variants.



A Modern-style sofa doesn't match Victorian-style chairs.

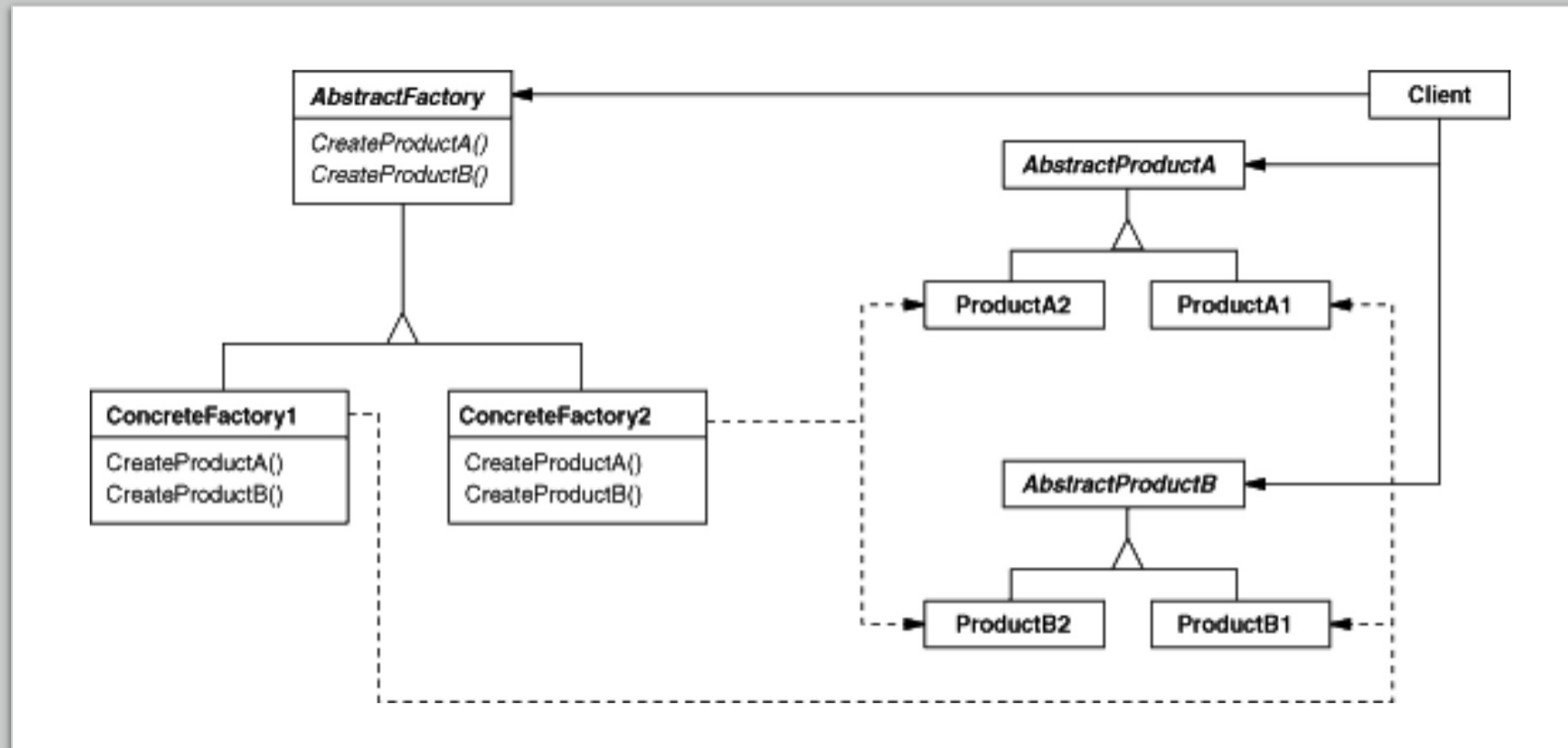
- Serve un modo per creare singoli oggetti di arredo in modo che corrispondano ad altri oggetti della stessa famiglia
 - Se i clienti ricevessero mobili non corrispondenti, potrebbero arrabbiarsi molto
- Inoltre, non si vuole modificare il codice esistente quando si aggiungono nuovi prodotti o famiglie di prodotti al programma
 - I fornitori di mobili aggiornano i loro cataloghi molto spesso e non vorresti cambiare il codice principale ogni volta che accade

Solution



All variants of the same object must be moved to a single class hierarchy.

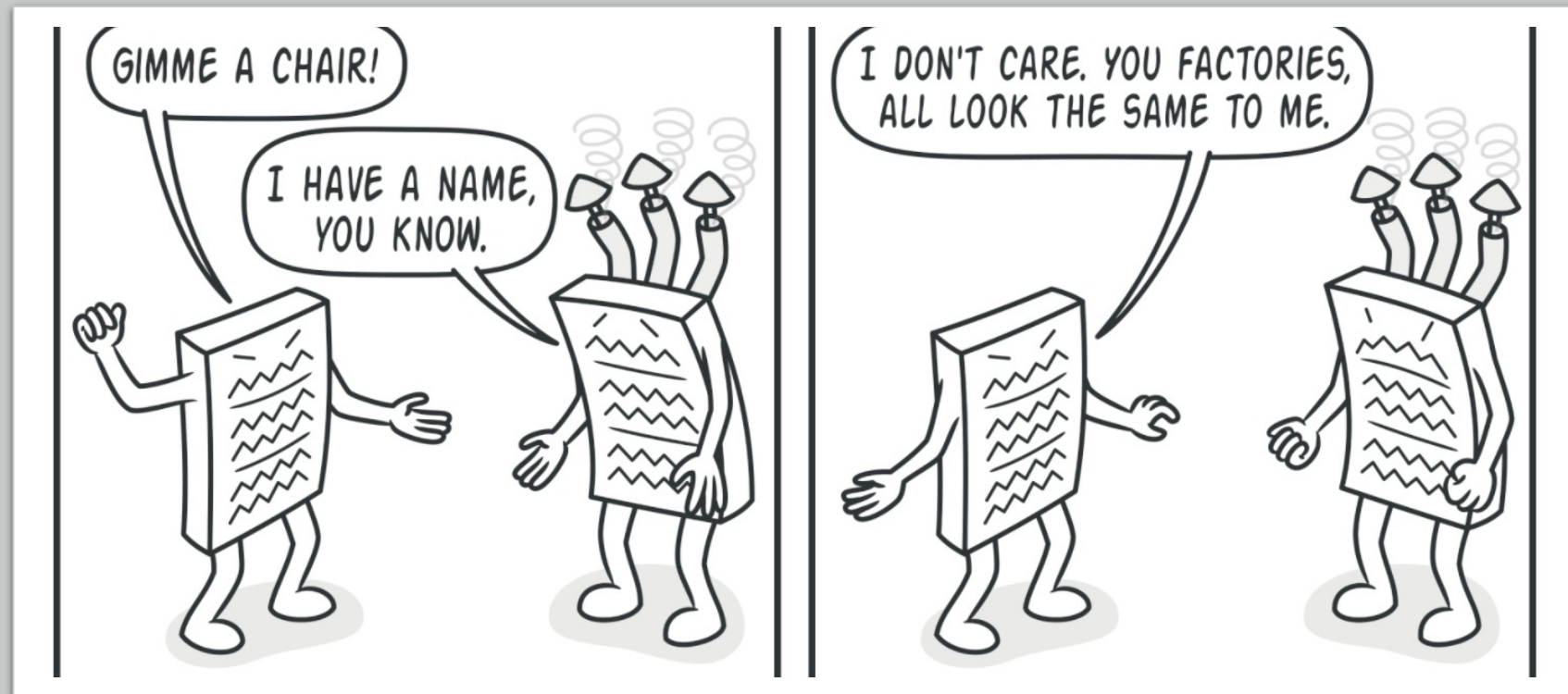
Each concrete factory corresponds to a specific product variant.



Abstract Factory

Struttura

- Il client deve funzionare sia con le factory che con i prodotti tramite le rispettive interfacce astratte
- Supponiamo che il client desideri una factory per produrre una sedia:
 - Il client non deve essere a conoscenza della classe della factory, né importa che tipo di sedia ottiene
 - Che si tratti di un modello moderno o di una sedia in stile vittoriano, il client deve trattare tutte le sedie allo stesso modo, utilizzando l'interfaccia astratta della sedia
 - Con questo approccio, l'unica cosa che il client sa della sedia è che implementa in qualche modo il metodo sitOn
 - Inoltre, qualunque variante di sedia venga restituita, corrisponderà sempre al tipo di divano o tavolino prodotto dallo stesso oggetto di fabbrica



Esempio di codice

```
/*  
 * GUIFactory example  
 */  
public abstract class GUIFactory {  
    public static GUIFactory getFactory() {  
        int sys = readFromConfigFile("OS_TYPE");  
        if (sys == 0) {  
            return new WinFactory();  
        } else {  
            return new OSXFactory();  
        }  
    }  
  
    public abstract Button createButton();  
}
```

Esempio di codice

```
class WinFactory extends GUIFactory {
    public Button createButton() {
        return new WinButton();
    }
}

class OSXFactory extends GUIFactory {
    public Button createButton() {
        return new OSXButton();
    }
}

public abstract class Button {
    public abstract void paint();
}

class WinButton extends Button {
    public void paint() {
        System.out.println("Sono un WinButton!");
    }
}

class OSXButton extends Button {
    public void paint() {
        System.out.println("Sono un OSXButton!");
    }
}
```

Esempio di codice

```
public class Application {  
    public static void main(String[] args) {  
        GUIFactory factory = GUIFactory.getFactory();  
        Button button = factory.createButton();  
        button.paint();  
    }  
    // L'output sarà:  
    //     "Sono un WinButton!"  
    // oppure:  
    //     "Sono un OSXButton!"  
}
```

Conseguenze di Abstract Factory

1. **Isola le classi concrete** Aiuta a controllare le classi di oggetti che un'applicazione crea poiché una factory racchiude la responsabilità e il processo di creazione degli oggetti del prodotto, isolando così i client dalle classi di implementazione
2. **Rende facile lo scambio di famiglie di prodotti** La concrete factory appare solo una volta in un'applicazione, ovvero dove viene istanziata. Ciò semplifica la modifica della concrete factory utilizzata da un'applicazione. Si possono utilizzare diverse configurazioni di prodotto semplicemente cambiando la concrete factory. Poiché una abstract factory crea una famiglia completa di prodotti, l'intera famiglia di prodotti cambia contemporaneamente
3. **Promuove la coerenza tra i prodotti** Quando gli oggetti del prodotto in una famiglia sono progettati per funzionare insieme, è importante che un'applicazione utilizzi gli oggetti di una sola famiglia alla volta. AbstractFactory rende facile da applicare questo concetto
4. **Supportare nuovi tipi di prodotti è difficile** Estendere le abstract factory per produrre nuovi tipi di prodotti non è facile, perché l'interfaccia AbstractFactory corregge l'insieme di prodotti che possono essere creati. Il supporto di nuovi tipi di prodotti richiede l'estensione dell'AbstractFactory, che implica la modifica della classe AbstractFactory e di tutte le sue sottoclassi

Singleton

Descrizione

- Assicura che una classe abbia una sola istanza, e fornisce un punto globale di accesso alla stessa

Motivazioni

- Un applicativo deve istanziare un oggetto che gestisce una stampante
- Questo oggetto deve essere unico, vale dire, deve esserci una sola istanza di esso, altrimenti potrebbero risultare dei problemi nella gestione della risorsa
- Il problema è la definizione di una classe che garantisca la creazione di un'unica istanza all'interno del programma

Caratteristiche



Definisce una classe della quale è possibile istanziare un unico oggetto, tramite l'invocazione a un metodo della classe, incaricato della produzione degli oggetti

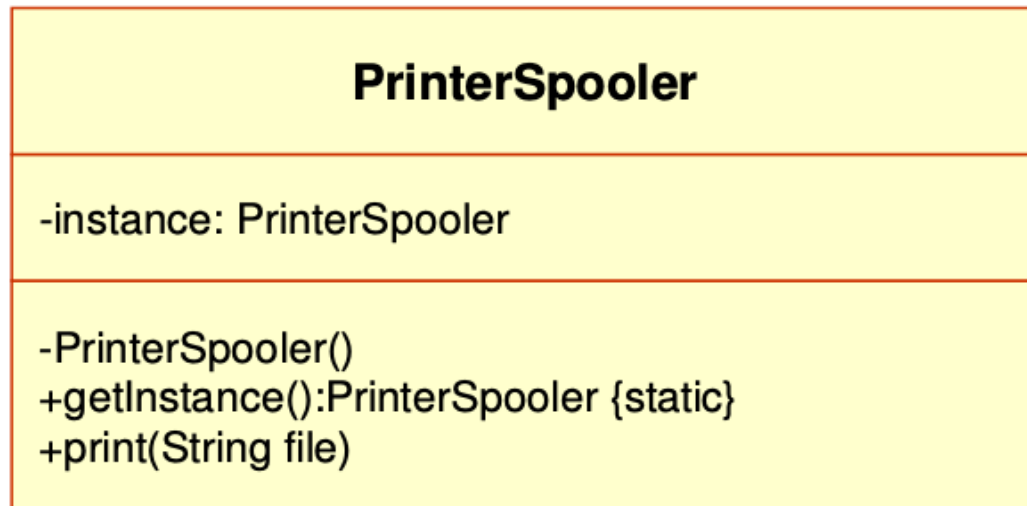
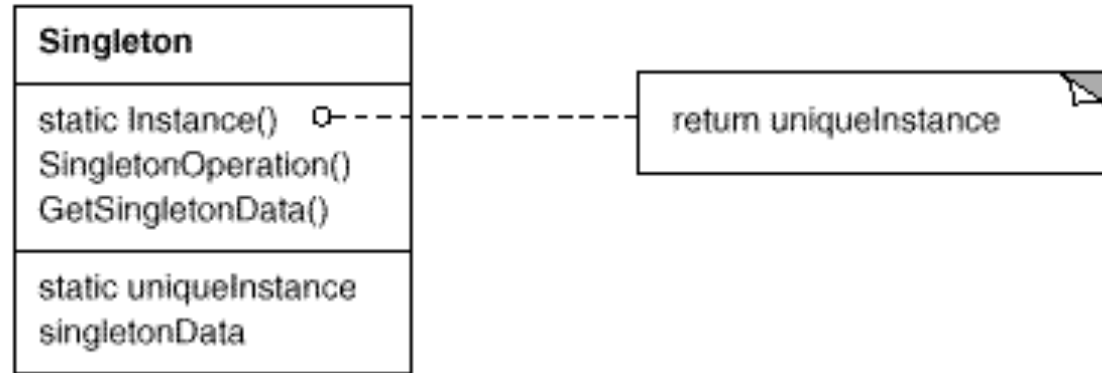


Diverse richieste di istanziare, comportano la restituzione di un riferimento allo stesso oggetto

Applicabilità

- Usa il pattern Singleton quando:
 - deve esserci esattamente un'istanza di una classe e deve essere accessibile ai client da un punto di accesso noto
 - l'unica istanza si deve poter estendere mediante sottoclassi e i client dovrebbero essere in grado di utilizzare un'istanza estesa senza modificare il proprio codice

Struttura



```
static PrinterSpooler getInstance(){
    if( instance == null)
        instance = new PrinterSpooler();
    return instance;
}
```

```
package com.devblog.designpatterns;

public class Demo {
    private static Demo instance = null;
    private Demo() {}
    public static Demo getInstance() {
        if (instance == null) {
            instance = new Demo();
        }
        return instance;
    }
}
```

Esempio di
codice

Conseguenze

- ***Accesso controllato all'istanza unica*** Poiché la classe Singleton incapsula la sua unica istanza, può avere un controllo rigoroso su come e quando i client vi accedono
- ***Spazio dei nomi ridotto*** Il modello Singleton è un miglioramento rispetto alle variabili globali. Evita di inquinare lo spazio dei nomi con variabili globali che memorizzano le sole istanze
- ***Permette l'affinamento delle operazioni e la rappresentazione*** La classe Singleton può essere sottoclasse ed è facile configurare un'applicazione con un'istanza di questa classe estesa. È possibile configurare l'applicazione con un'istanza della classe necessaria in fase di esecuzione
- ***Consente un numero variabile di istanze*** Il modello rende facile cambiare idea e consente più di un'istanza della classe Singleton. Inoltre, è possibile utilizzare lo stesso approccio per controllare il numero di istanze utilizzate dall'applicazione. È necessario modificare solo l'operazione che **concede l'accesso all'istanza Singleton**

Esercizio

- Vendita di sistemi Hi-Fi, dove si eseguono dimostrazioni dell'utilizzo di famiglie complete di prodotti
- Due famiglie di prodotti, basate su tecnologie diverse:
 - come supporto il nastro (tape)
 - come supporto il compact disc
- In entrambi casi, ogni famiglia è composta da:
 - Il supporto stesso (tape o cd)
 - un masterizzatore (recorder)
 - un riproduttore (player)
- Se si accetta il fatto che questi prodotti offrono agli utenti una stessa interfaccia (cosa non tanto distante dalla realtà), un cliente potrebbe essere in grado di eseguire lo stesso processo di prova su prodotti di entrambe famiglie di prodotti
 - Ad esempio, potrebbe eseguire prima una registrazione nel recorder, per poi dopo ascoltarla nel player
- Il problema consiste nella definizione di un modo di creare famiglie complete di prodotti, senza vincolare il codice delle particolari famiglie alla codifica del client che li utilizza



Riferimenti

- <https://refactoring.guru/design-patterns/>
- <http://eii.pucv.cl/pers/guidi/documentos/Guidi-GoFDesignPatternsInJava.pdf>
- <https://www.cs.unc.edu/~stotts/GOF/hires/patcafsa.htm>